Week 2 - DAY-2 Hands-On Activities Keerthivasan R

PROBLEM 1: Student Marks Analyzer (LEVEL-1)

Scenario:

You are developing a small utility for a teacher to analyze student marks stored in an array.

📌 Requirements

- Store student marks in an array.

- Calculate total and average marks using array methods.

- Display pass/fail result based on average.

- Use let/const appropriately.

- Use arrow functions for calculations.

- Display output using template literals.

🛠️ Technical Constraints

- Must use array methods like reduce() and map().

- Use only ES6+ syntax.

- No external libraries.

- All logic must be inside modular JavaScript file.

🎯 Learning Outcome

- Understand array processing.

- Practice ES6 features.

- Learn modular coding structure.

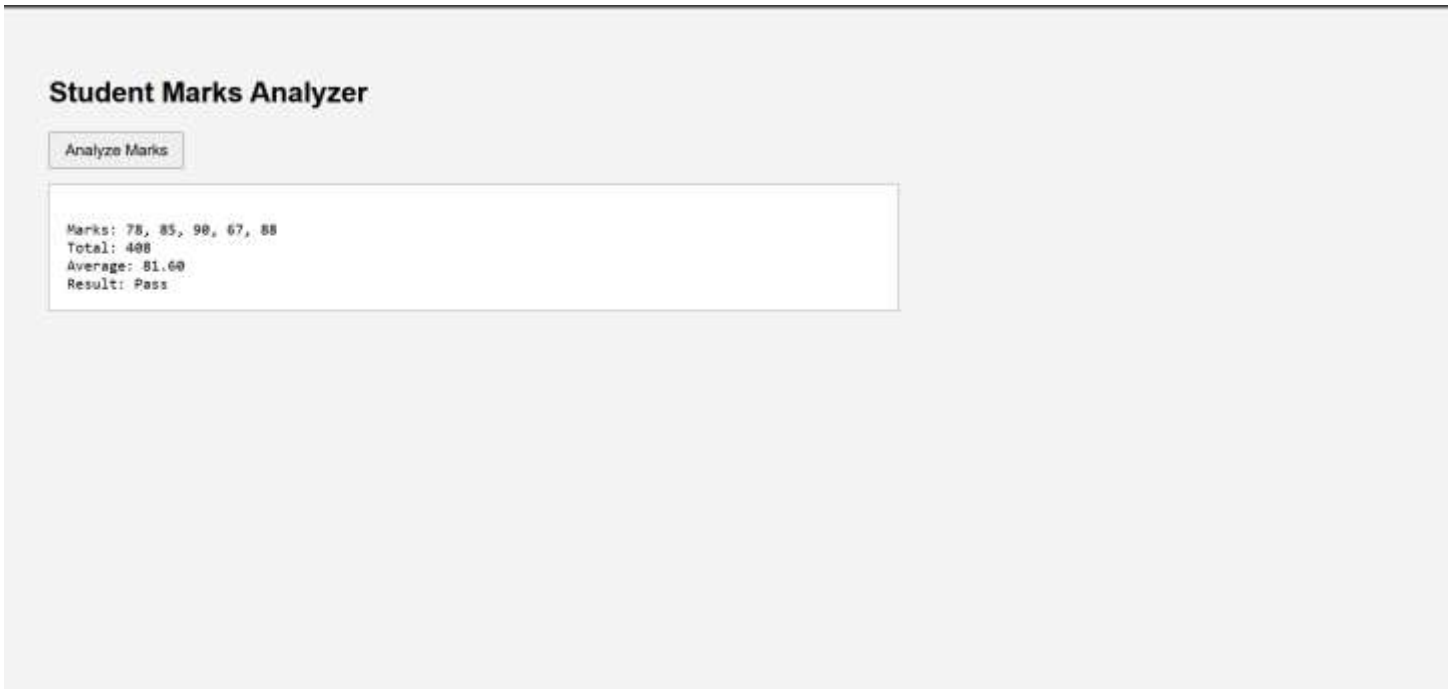
CODE:

```
<!DOCTYPE html>
<html>
```

```
<head>
    <title>Student Marks Analyzer</title>
    <style>
        body {
            font-family: Arial;
            background-color: #f4f4f4;
            padding: 30px;
        }

        h2 {
            margin-bottom: 20px;
        }

        button {
            padding: 6px 12px;
            cursor: pointer;
        }

        pre {
            background: white;
            padding: 15px;
            width: 60%;
            border: 1px solid #999;
        }
    </style>
</head>

<body>

    <h2>Student Marks Analyzer</h2>

    <button onclick="analyzeMarks()">Analyze Marks</button>

    <pre id="output"></pre>

    <script>
        // Array storing student marks
        const marks = [78, 85, 90, 67, 88];

        /*
          Arrow function to calculate total marks
          reduce() loops through array and accumulates sum
          sum → accumulator
          mark → current value
          0 → initial value
        */
        const calculateTotal = arr =>
            arr.reduce((sum, mark) => sum + mark, 0);
```

```
        // Calculates average using total / length
        const calculateAverage = arr =>
            calculateTotal(arr) / arr.length;

        // Ternary operator to determine pass/fail
        const getResult = avg =>
            avg >= 50 ? "Pass" : "Fail";

        // Main function triggered by button click
        const analyzeMarks = () => {

            const total = calculateTotal(marks);
            const average = calculateAverage(marks);
            const result = getResult(average);

            // Display output using template literals
            document.getElementById("output").textContent = `
Marks: ${marks.join(", ")}
Total: ${total}
Average: ${average.toFixed(2)}
Result: ${result}
`;
        };
    </script>

</body>

</html>
```

OUTPUT:



PROBLEM 2: Product Cart Summary (LEVEL-1)

Scenario:

Build a simple shopping cart summary system.

📌 Requirements

- Store product objects (name, price, quantity) in an array.
- Calculate total cart value.
- Display formatted invoice using template literals.
- Use arrow functions.
- Export calculation function from a module.

🛠️ Technical Constraints

- Use map() and reduce().
- Use ES6 modules (export/import).
- No DOM manipulation required.

🎯 Learning Outcome

- Work with arrays of objects.
- Understand modules.
- Improve functional programming skills.

CODE:

```
<!DOCTYPE html>
<html>

<head>
    <title>Product Cart Summary</title>
    <style>
        body {
            font-family: Arial;
            background-color: #f4f4f4;
            padding: 30px;
        }
```

```
        h2 {
            margin-bottom: 20px;
        }

        button {
            padding: 6px 12px;
            cursor: pointer;
        }

        pre {
            background: white;
            padding: 15px;
            width: 60%;
            border: 1px solid #999;
        }
    </style>
</head>

<body>

    <h2>Shopping Cart Summary</h2>

    <button onclick="generateInvoice()">Generate Invoice</button>

    <pre id="invoice"></pre>

    <script>
        // Array of product objects
        const products = [
            { name: "Laptop", price: 50000, quantity: 1 },
            { name: "Mouse", price: 500, quantity: 2 },
            { name: "Keyboard", price: 1500, quantity: 1 }
        ];

        /*
          Calculate total cart value
          price * quantity for each product
        */
        const calculateCartTotal = items =>
            items.reduce((total, item) =>
                total + item.price * item.quantity, 0);

        const generateInvoice = () => {

            const total = calculateCartTotal(products);

            // map() creates formatted product list
            document.getElementById("invoice").textContent = `
${products.map(p =>
```

```
                `${p.name} - ₹${p.price} x ${p.quantity}`
            ).join("\n")}

Total Amount: ₹${total}
`;
        };
    </script>

</body>

</html>
```

OUTPUT:

PROBLEM 3: Weather Data Fetcher (LEVEL-2)

Scenario:

Create an application that fetches weather data asynchronously.

📌 **Requirements**

- Fetch data from a public API using fetch().

- Implement version using Promises.

- Implement version using async/await.

- Handle errors properly.

- Display formatted weather report.

🛠️ Technical Constraints

- Use async/await.

- Use template literals.

- Use arrow functions.

- Must include proper error handling using try/catch.

🎯 Learning Outcome

- Understand Promises and async/await.

- Handle asynchronous operations.

- Improve API handling skills.

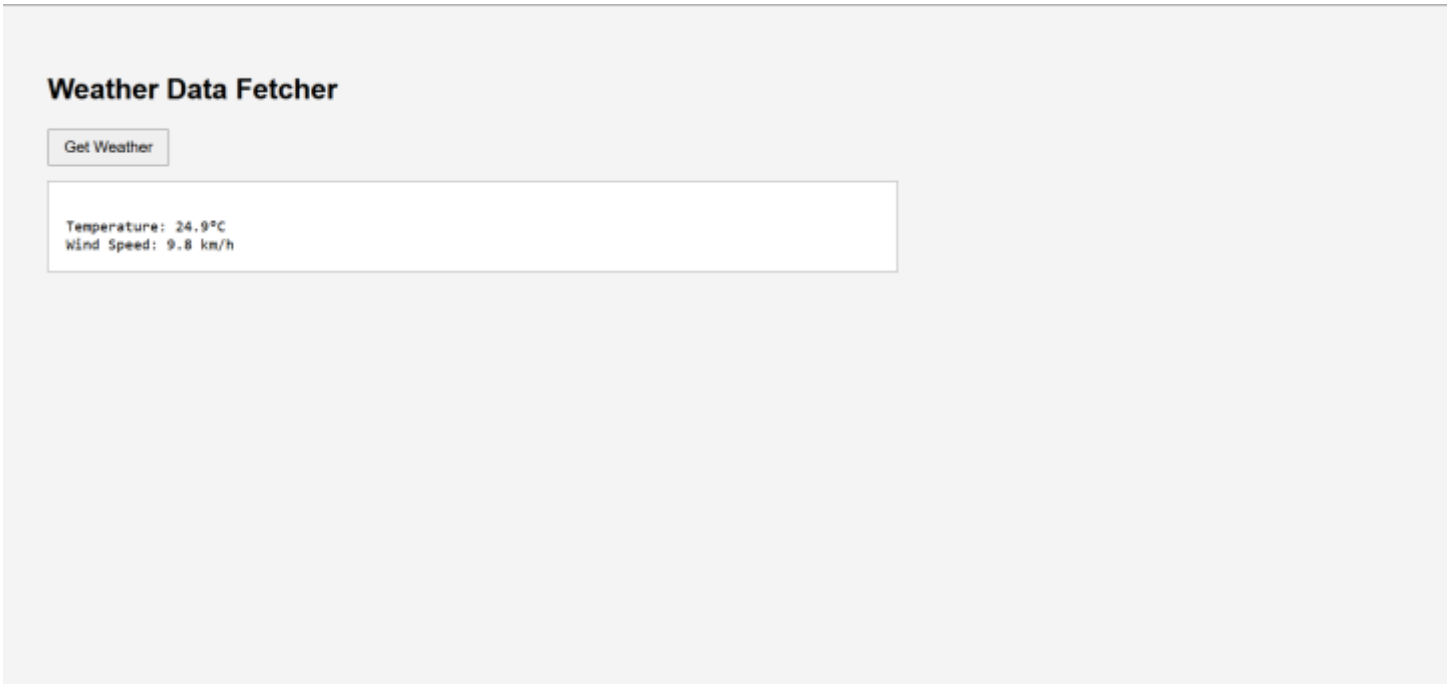
CODE:

```
<!DOCTYPE html>
<html>

<head>
    <title>Weather Data Fetcher</title>
    <style>
        body {
            font-family: Arial;
            background-color: #f4f4f4;
            padding: 30px;
        }

        h2 {
            margin-bottom: 20px;
        }

        button {
            padding: 6px 12px;
            cursor: pointer;
        }

        pre {
            background: white;
            padding: 15px;
```

```
            width: 60%;
            border: 1px solid #999;
        }
    </style>
</head>

<body>

    <h2>Weather Data Fetcher</h2>

    <button onclick="fetchWeather()">Get Weather</button>

    <pre id="weatherOutput"></pre>

    <script>
        // API endpoint (Hyderabad location)
        const API_URL =
            "https://api.open-
meteo.com/v1/forecast?latitude=17.3850&longitude=78.4867&current_weather=true";

        /*
          Async function allows use of await
          await pauses execution until Promise resolves
        */
        const fetchWeather = async () => {
            try {

                // Fetch data from API
                const response = await fetch(API_URL);

                // Check if response is successful
                if (!response.ok) {
                    throw new Error("Failed to fetch data");
                }

                // Convert response to JSON
                const data = await response.json();

                // Display weather details
                document.getElementById("weatherOutput").textContent = `
Temperature: ${data.current_weather.temperature}°C
Wind Speed: ${data.current_weather.windspeed} km/h
`;

            } catch (error) {
                // Handle errors
                document.getElementById("weatherOutput").textContent =
                    "Error: " + error.message;
            }
        };
```

```
    </script>

</body>

</html>
```

OUTPUT:



PROBLEM 4: Task Manager with Async Storage (LEVEL-2)

Scenario:

Develop a task manager where tasks are saved and retrieved asynchronously.

📌 Requirements

- Store tasks in an array.

- Create addTask(), deleteTask(), and listTasks() functions.

- Simulate async storage using setTimeout with callbacks.

- Convert callback-based logic into Promises.

- Refactor into async/await version.

- Use ES6 modules.

🛠️ Technical Constraints

- Must demonstrate callback → promise → async/await evolution.

- Use let/const properly.

- Use arrow functions.

- Use template literals for display.

🎯 Learning Outcome

- Deep understanding of async JavaScript.

- Learn refactoring async code.

- Master modern ES6+ features.

CODE:

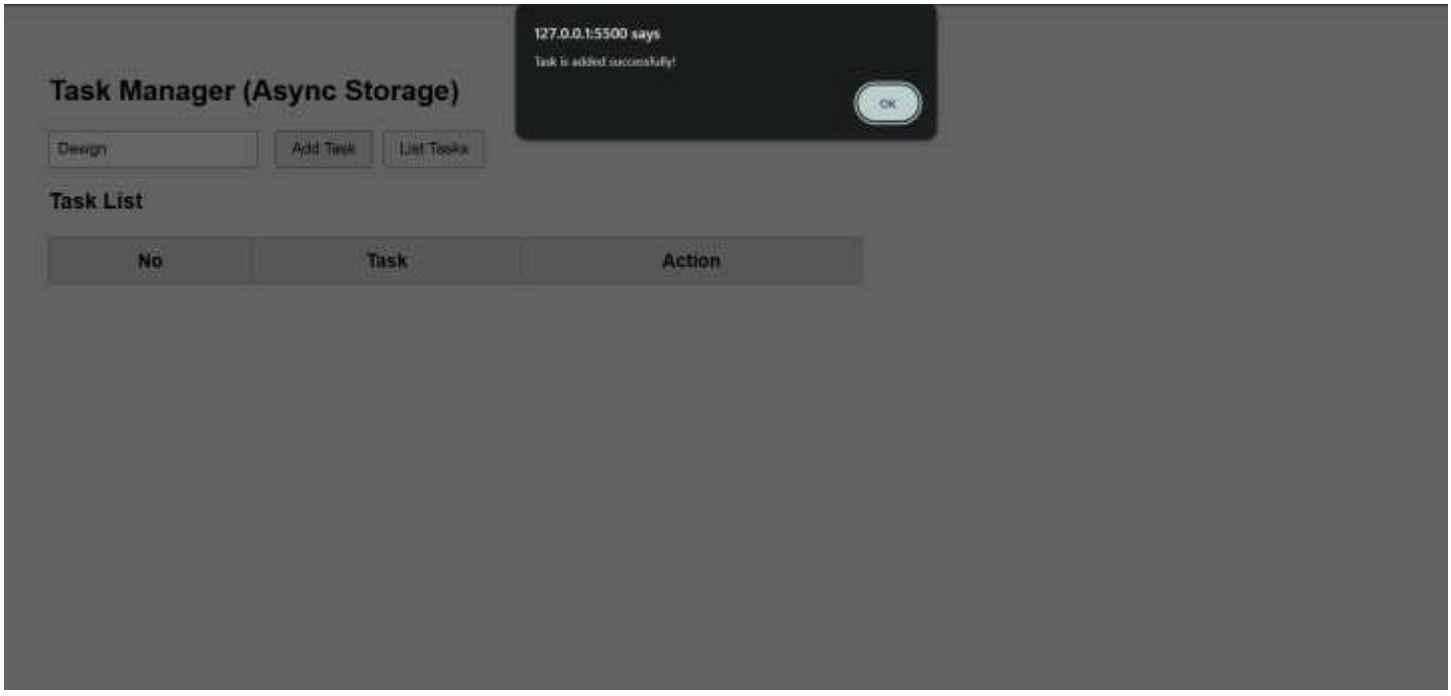
```
<!DOCTYPE html>
<html>

<head>
    <title>Task Manager</title>
    <style>
        body {
            font-family: Arial;
            background-color: #f4f4f4;
            padding: 30px;
        }

        h2 {
            margin-bottom: 20px;
        }

        input {
            padding: 6px;
            margin-right: 10px;
        }

        button {
            padding: 6px 12px;
            cursor: pointer;
```

```
            margin-right: 5px;
        }

        table {
            margin-top: 20px;
            border-collapse: collapse;
            width: 60%;
            background: white;
        }

        th,
        td {
            border: 1px solid #999;
            padding: 10px;
            text-align: center;
        }

        th {
            background-color: #ddd;
        }
    </style>
</head>

<body>

    <h2>Task Manager (Async Storage)</h2>

    <input type="text" id="taskInput" placeholder="Enter task">
    <button onclick="addTask()">Add Task</button>
    <button onclick="listTasks()">List Tasks</button>

    <h3>Task List</h3>

    <table id="taskTable">
        <thead>
            <tr>
                <th>No</th>
                <th>Task</th>
                <th>Action</th>
            </tr>
        </thead>
        <tbody></tbody>
    </table>

    <script>

        // Array to store tasks in memory
        let tasks = [];

        /*
```

```
  Add task (simulated async using setTimeout)
  Mimics database delay
*/
const addTask = () => {

    const task = document.getElementById("taskInput").value;

    if (!task) {
        alert("Please enter a task");
        return;
    }

    setTimeout(() => {
        tasks.push(task);  // Add task to array
        alert("Task is added successfully!");
        document.getElementById("taskInput").value = "";
    }, 500);
};

/*
  Delete task using index
  splice() removes element from array
*/
const deleteTask = (index) => {

    setTimeout(() => {
        tasks.splice(index, 1);
        alert("Task deleted successfully!");
        listTasks(); // Refresh UI
    }, 500);
};

/*
  Display tasks in table
*/
const listTasks = () => {

    const tbody = document.querySelector("#taskTable tbody");
    tbody.innerHTML = "";

    // If no tasks available
    if (tasks.length === 0) {
        tbody.innerHTML = `
    <tr>
        <td colspan="3">No tasks available</td>
    </tr>
`;
        return;
    }
```

```
            // Loop through tasks using forEach
            tasks.forEach((task, index) => {

                tbody.innerHTML += `
            <tr>
                <td>${index + 1}</td>
                <td>${task}</td>
                <td>
                    <button onclick="deleteTask(${index})">
                        Delete
                    </button>
                </td>
            </tr>
        `;
            });
        };

    </script>

</body>

</html>
```

OUTPUT:



Task Manager (Async Storage)

Enter task | Add Task | List Tasks

Task List

No	Task	Action
1	Design	Delete